

Redux and Firestore Study Notes

Project: Warhammer40KApp

Goal: explain how Redux, slices, reducers, dispatch, Firestore, and the related design patterns work in this app.

Important defense wording: if Redux or Firestore was required by the assignment, do not say "I chose it." Say "The requirement was to use it, and I implemented it in this structure."

Table of Contents

1. Redux: Store, Slices, Reducers, Actions, Dispatch, and Selectors
2. Firestore: Collections, Documents, Reads, Writes, Listeners, and Patterns
3. Firebase Authentication and How It Connects To Firestore
4. How Redux and Firestore Work Together In This App
5. Short Defense Answers

1. Redux: Store, Slices, Reducers, Actions, Dispatch, and Selectors

What Redux is

Redux is a central state container. Instead of every screen keeping its own separate copy of important shared data, Redux stores that shared data in one place.

In this app, Redux mainly stores the user's army compositions.

```
Component presses button
  -> dispatch(action)
  -> Redux store receives action
  -> reducer decides how state changes
  -> components using that state re-render
```

The store

The Redux store is created in `src/store/index.ts`.

```
const rootReducer = combineReducers({
  armyCompositions: armyCompositionsReducer,
});
```

This means the global Redux state has a property named `armyCompositions`. That property is controlled by the army compositions reducer.

```
{
  armyCompositions: []
}
```

The store is made available to the whole app in `App.tsx`:

```
<Provider store={store}>
  ...
</Provider>
```

Without this provider, components could not use `useAppDispatch` or `useAppSelector`, because React would not know which Redux store to use.

The slice

A slice is Redux Toolkit's way to group one feature's state, reducers, and generated actions.

Your slice is in `src/features/armyCompositions/armyCompositionSlice.ts`.

```
const initialState: ArmyComposition[] = [];
```

This says the army compositions state starts as an empty array.

```
export const armyCompositionSlice = createSlice({
  name: "armyCompositions",
  initialState,
  reducers: {
    armyCompositionAdded(state, action: { payload: ArmyComposition }) {
      state.push(action.payload);
    },
  },
});
```

`createSlice` does several things:

- It gives the feature a name: `armyCompositions`.
- It stores the initial state.
- It defines reducer functions.
- It automatically creates action creator functions with the same names as the reducers.

Reducers

A reducer is a function that changes Redux state based on an action.

Example:

```
armyCompositionAdded(state, action: { payload: ArmyComposition }) {
  state.push(action.payload);
}
```

This means: when Redux receives the `armyCompositionAdded` action, take the new army composition from `action.payload` and add it to the array.

Why `state.push` is allowed

In classic Redux, reducers are not allowed to directly mutate state. But this project uses Redux Toolkit, and Redux Toolkit uses Immer internally.

Immer lets you write code that looks like mutation:

```
state.push(action.payload);
```

But internally, it still creates a safe immutable state update.

Defense answer: Redux Toolkit allows simpler reducer syntax through Immer. Although the code looks like direct mutation, Redux Toolkit safely creates the new immutable state behind the scenes.

Actions and payloads

An action is an object that describes what happened. A payload is the data attached to the action.

This code in your app:

```
dispatch(armyCompositionAdded(newArmyComposition));
```

creates an action similar to this:

```
{
  type: "armyCompositions/armyCompositionAdded",
  payload: newArmyComposition
}
```

The `type` tells Redux what happened. The `payload` contains the new data needed by the reducer.

Dispatch

`dispatch` sends an action to Redux.

```
dispatch(armyCompositionAdded(newArmyComposition));
```

This means: "Hey Redux, an army composition was added. Here is the new army composition. Please update the store."

Redux receives the action, checks the action type, and runs the matching reducer from the slice.

Selectors

A selector reads data from Redux state.

```
export const selectArmyCompositions = (state: RootState) =>
  state.armyCompositions;
```

This selector says: from the full Redux state, return only the army compositions part.

The app reads that data with:

```
const armyCompositions = useAppSelector(selectArmyCompositions);
```

When the Redux state changes, components using this selector automatically re-render with the new data.

Full Redux example from the app

User creates army

- > useCreateArmyComposition creates newArmyComposition
- > dispatch(armyCompositionAdded(newArmyComposition))
- > Redux reducer adds it to the armyCompositions array
- > HomeScreen reads updated Redux state
- > new army card appears
- > Firestore also saves it remotely

Important Redux vocabulary

- **Store:** the central state container.
- **Slice:** one feature's Redux state and reducers.
- **Reducer:** a function that updates state based on an action.
- **Action:** an object describing what happened.
- **Payload:** the data sent with the action.
- **Dispatch:** the function used to send an action to Redux.
- **Selector:** a function used to read data from Redux.

Defense answer: The requirement was to use Redux. In this app, Redux stores army compositions because that data is shared between screens. The slice defines the allowed state changes, dispatch sends actions, reducers update the state, and selectors let screens read the updated data.

2. Firestore: Collections, Documents, Reads, Writes, Listeners, and Patterns

What Firestore is

Firestore is a NoSQL cloud database from Firebase. It stores data in collections and documents instead of SQL tables and rows.

Traditional SQL databases use:

```
tables
rows
columns
relations
```

Firestore uses:

```
collections
documents
fields
subcollections
```

Firestore data model

A collection is a group of documents. A document is like a JavaScript object with fields.

```
{
  id: "abc",
  name: "My Army",
  totalPoints: 500
}
```

A subcollection is a collection inside a document.

Firestore in your app

The Firebase setup is in `src/config/firebase.ts`:

```
const app = initializeApp(firebaseConfig);

export const auth = getAuth(app);
export const database = getFirestore(app);
```

`initializeApp` connects the React Native app to the Firebase project. `auth` is the Firebase Auth service. `database` is the Firestore database connection.

Service layer pattern

Firestore functions are placed in `src/services/firestoreService.ts`.

This is a service layer. Screens do not directly contain Firestore calls. Instead, they call service functions like:

- `subscribeToArrayCompositions`
- `saveArmyComposition`
- `updateArmyCompositionUnits`
- `deleteArmyComposition`
- `getArmies`

Defense answer: Firestore access is isolated in a service file so the screens do not directly depend on Firebase implementation details.

Your Firestore structure

User-created army compositions are stored like this:

```
users/{userId}/armyCompositions/{armyCompositionId}
```

In code, that looks like this:

```
doc(database, "users", userId, "armyCompositions", armyComposition.id)
```

Example structure:

```
users
  alexUserId
    armyCompositions
      army1
      army2

  otherUserId
    armyCompositions
      army3
```

This is a user-owned data pattern. Each authenticated user has their own army compositions subcollection.

Defense answer: The app stores army compositions under the authenticated user's id. That keeps each user's data logically separated and makes security rules easier because access can be checked against the user id.

collection versus doc

This points to a collection:

```
collection(database, "users", userId, "armyCompositions")
```

This points to one specific document:

```
doc(database, "users", userId, "armyCompositions", armyComposition.id)
```

`collection(...)` means "a list of documents." `doc(...)` means "one specific document."

Reading data once with `getDocs`

The app uses `getDocs` to load the available armies:

```
const querySnapshot = await getDocs(collection(database, "armies"));
```

`getDocs` reads data once. In this app, that is used for shared army reference data.

Defense answer: The app uses `getDocs` for armies because armies are shared reference data and do not need constant real-time updates while the user is on the screen.

Listening in real time with `onSnapshot`

The app uses `onSnapshot` for user army compositions:

```
return onSnapshot(
  collection(database, "users", userId, "armyCompositions"),
  (querySnapshot) => {
    const armyCompositions = querySnapshot.docs.map((document) => {
      return document.data() as ArmyComposition;
    });

    onArmyCompositionsChanged(armyCompositions);
  },
);
```

`onSnapshot` keeps listening. When Firestore data changes, Firebase sends a new snapshot to the app.

This is the observer or subscription pattern. The app subscribes to changes, and Firestore notifies the app when data changes.

Defense answer: The app uses Firestore's real-time listener so local state stays synchronized with the database. When Firestore changes, the listener receives a snapshot and dispatches the new data into Redux.

Saving data with `setDoc`

This saves a full army composition:

```
return setDoc(  
  doc(database, "users", userId, "armyCompositions", armyComposition.id),  
  armyComposition,  
);
```

`doc(...)` creates a reference to the document. `setDoc(...)` writes the data to that document. If the document does not exist, Firestore creates it. If it exists, Firestore replaces it unless merge is used.

Updating part of a document with merge

```
return setDoc(  
  doc(database, "users", userId, "armyCompositions", armyCompositionId),  
  {  
    units,  
    totalPoints,  
  },  
  { merge: true },  
);
```

`{ merge: true }` means update only the provided fields. In this case, Firestore updates `units` and `totalPoints`, while keeping fields like `id`, `name`, and `army`.

Defense answer: `merge: true` is used so updating units does not overwrite the whole army composition document.

Deleting data

```
return deleteDoc(  
  doc(database, "users", userId, "armyCompositions", armyCompositionId),  
);
```

`deleteDoc` removes one specific document from Firestore.

Important Firestore patterns in this app

- **User-owned data pattern:** each user's data is stored under `users/{userId}`.
- **Service layer pattern:** Firestore logic lives in service functions, not directly in screens.
- **Observer/subscription pattern:** `onSnapshot` listens for real-time database changes.
- **Optimistic UI pattern:** the app sometimes updates Redux before Firestore finishes saving, so the UI feels faster.
- **Query-shaped data pattern:** Firestore documents are shaped around how the app reads them, because Firestore is not built around SQL joins.

- **Async data pattern:** reads and writes use `async` and `await` because database work takes time.

3. Firebase Authentication and How It Connects To Firestore

Important distinction

Firebase Authentication and Firestore are related, but they are not the same thing.

- **Firebase Authentication** proves who the user is. It handles register, login, logout, password updates, and email updates.
- **Firestore** stores database documents, such as armies and army compositions.

Authentication answers the question: "Who is using the app?" Firestore answers the question: "What data should be stored or loaded?"

Firebase setup

In `src/config/firebase.ts`, the app creates both an Auth instance and a Firestore instance:

```
const app = initializeApp(firebaseConfig);

export const auth = getAuth(app);
export const database = getFirestore(app);
```

`auth` is the object used by Firebase Authentication functions. `database` is the object used by Firestore functions.

Why auth is passed as the first parameter

Firebase uses the modular SDK style. In this style, most Firebase functions receive the service instance as their first argument.

For example, in `src/services/authService.ts`:

```
export const login = (auth: Auth, email: string, password: string) => {
  return signInWithEmailAndPassword(auth, email, password);
};
```

The call looks like this:

```
signInWithEmailAndPassword(auth, email, password)
```

The first parameter, `auth`, tells Firebase which Firebase Auth instance should handle this login request. The email and password are the actual credentials being checked.

This is similar to Firestore functions using the database instance:

```
getDocs(collection(database, "armies"))
```

Here, `database` tells Firebase which Firestore database to read from.

Defense answer: Firebase's modular API passes the service instance first. For authentication functions, that first parameter is `auth`. For Firestore functions, it is usually `database` or a document/collection reference created from `database`.

Why the service functions receive `auth` as a parameter

Your service functions are written like this:

```
export const register = (auth: Auth, email: string, password: string) => {  
  return createUserWithEmailAndPassword(auth, email, password);  
};
```

They could have imported `auth` directly inside the service file, but this version passes `auth` in from the screen:

```
await login(auth, email, password);
```

This makes the service function explicit: it needs an `Auth` instance, an email, and a password. It also makes the function easier to test or reuse with another `Auth` instance.

Register flow

When the user registers, the app calls:

```
await register(auth, email, password);
```

The service calls:

```
createUserWithEmailAndPassword(auth, email, password)
```

Firebase creates the account. After registration, Firebase automatically signs in the new user. That login state change is detected by `onAuthStateChanged` in `AuthContext`.

Login flow

When the user logs in, the app calls:

```
await login(auth, email, password);
```

The service calls:

```
signInWithEmailAndPassword(auth, email, password)
```

Firebase checks the email and password. If they are correct, Firebase updates the auth state. Then `AuthContext` receives the logged-in user.

AuthContext watches login state

The important listener is in `src/contexts/AuthContext.tsx`:

```
const unsubscribe = onAuthStateChanged(auth, (user) => {  
  setCurrentUser(user);  
  setLoading(false);  
});
```

`onAuthStateChanged` watches Firebase Auth. It runs when:

- the user logs in
- the user logs out
- the app reloads and Firebase checks whether a saved session still exists

If a user is logged in, `user` is a Firebase `User` object. If no user is logged in, `user` is `null`.

How authentication connects to Firestore

After login, Firebase gives the app a user object. The most important field for Firestore paths is:

```
currentUser.uid
```

The `uid` is Firebase's unique id for that authenticated user. Your app uses it in Firestore paths:

```
collection(database, "users", userId, "armyCompositions")
```

When the hook calls the service, it passes the logged-in user's uid:

```
subscribeToArmyCompositions(  
  database,  
  currentUser.uid,  
  (armyCompositions) => {  
    dispatch(armyCompositionsLoaded(armyCompositions));  
  },  
);
```

This creates a path like:

```
users/{currentUser.uid}/armyCompositions
```

So the logged-in user's id decides which Firestore subcollection is loaded.

Defense answer: Firebase Auth identifies the user. The app then uses `currentUser.uid` to

read and write that user's Firestore data under `users/{uid}` .

Firestore security rules concept

In the app code, using `currentUser.uid` creates the correct path. But real protection should also happen in Firestore security rules.

A typical rule concept is:

```
allow read, write: if request.auth != null
  && request.auth.uid == userId;
```

This means a user may only access documents where the path's `userId` matches their authenticated Firebase uid.

The frontend path is important for organization. Firestore rules are important for security.

Defense answer: The frontend uses the authenticated user's uid to build the correct Firestore path. Firestore security rules should enforce that users can only access their own documents.

Why sensitive account changes reauthenticate

Changing email or password is sensitive. Firebase often requires recent authentication before allowing those operations.

```
const credential = EmailAuthProvider.credential(user.email, password);

await reauthenticateWithCredential(user, credential);
```

This proves that the person using the app still knows the current password. After that, Firebase allows sensitive operations like updating email or password.

```
await verifyBeforeUpdateEmail(user, newEmail);
```

This sends a verification email before changing to the new email address.

```
return updatePassword(user, newPassword);
```

This updates the Firebase Auth password after reauthentication succeeds.

Logout flow

Logout calls:

```
await logout(auth);
```

The service calls:

```
signOut(auth)
```

Firebase signs the user out. Then `onAuthStateChanged` receives `null`, `currentUser` becomes `null`, and `App.tsx` switches back to the auth screens.

Defense summary: Firebase Authentication handles identity. The app passes the `auth` instance to Firebase Auth functions because that is how the modular SDK knows which Auth service to use. Once a user is authenticated, their `uid` is used to load and save their Firestore data in a user-owned path.

4. How Redux and Firestore Work Together In This App

Firestore is the remote database. Redux is the local app state. React Native screens display the Redux state.

```
Firestore
  -> onSnapshot
  -> useLoadArmyCompositions hook
  -> dispatch
  -> Redux store
  -> selector
  -> screen UI
```

Loading army compositions

The hook `useLoadArmyCompositions` listens to Firestore when a user is logged in.

```
const unsubscribe = subscribeToArmyCompositions(
  database,
  currentUser.uid,
  (armyCompositions) => {
    dispatch(armyCompositionsLoaded(armyCompositions));
  },
);
```

This means Firestore sends the latest army compositions, and the hook dispatches them into Redux.

Creating an army composition

In `useCreateArmyComposition`, the app creates a new object, updates Redux, saves to Firestore, and navigates to the detail screen.

```
dispatch(armyCompositionAdded(newArmyComposition));

await saveArmyComposition(database, currentUser.uid, newArmyComposition);
```

Redux updates immediately. Firestore saves the data remotely.

Deleting an army composition

In `useArmyCompositions`, the app removes the composition locally first, then deletes it from Firestore.


```
dispatch(armyCompositionDeleted(id));

if (currentUser) {
  await deleteArmyComposition(database, currentUser.uid, id);
}
```

This is an optimistic update. The UI changes immediately instead of waiting for the network.

Updating units

In `useArmyCompositionUnits`, one helper updates three places:

- local screen state with `setSelectedUnits`
- Redux state with `dispatch`
- Firestore with `updateArmyCompositionUnits`

```
setSelectedUnits(updatedUnits);

dispatch(
  armyCompositionUnitsUpdated({
    armyCompositionId: armyComposition.id,
    units: updatedUnits,
    totalPoints: updatedTotalPoints,
  }),
);

await updateArmyCompositionUnits(
  database,
  currentUser.uid,
  armyComposition.id,
  updatedUnits,
  updatedTotalPoints,
);
```

Defense answer: Firestore stores the persistent remote data. Redux stores the local app state that screens read from. Hooks connect them by listening to Firestore and dispatching updates into Redux.

5. Short Defense Answers

How does Redux work in this app?

Redux stores army compositions in a central store. The army composition slice defines reducers for loading, adding, updating, clearing, and deleting compositions. Components dispatch actions to request state changes, reducers update the state, and selectors let screens read the updated state.

What is a slice?

A slice is one feature's Redux state and reducer logic grouped together. Redux Toolkit also generates action creators from the reducer names.

What is dispatch?

`dispatch` sends an action to the Redux store. The store then runs the matching reducer to update state.

What is a reducer?

A reducer is a function that receives the current state and an action, then produces the next state.

What is a payload?

The payload is the data sent with an action. For example, when adding an army composition, the payload is the new army composition object.

How does Firestore work in this app?

Firestore stores shared armies in a top-level `armies` collection and user-created army compositions under `users/{userId}/armyCompositions`. The app uses `getDocs` for one-time reads, `onSnapshot` for real-time user data, `setDoc` for saving and updating, and `deleteDoc` for deleting.

How does Firebase Authentication work in this app?

Firebase Authentication handles register, login, logout, and account updates. `AuthContext` listens to Firebase with `onAuthStateChanged`. When Firebase detects a logged-in user, the app stores that user in `currentUser`; when the user logs out, `currentUser` becomes `null`.

Why is auth passed as the first parameter?

Firebase's modular SDK passes the service instance first. Authentication functions receive `auth` first so Firebase knows which Auth instance should handle the operation. Firestore functions similarly use the `database` instance or references created from it.

How does authentication connect to Firestore?

After login, Firebase gives the app a user object with a unique `uid`. The app uses that uid in Firestore paths like `users/{uid}/armyCompositions`, so each user loads and saves their own army compositions.

What design patterns are used around Firestore?

The app uses a service layer, user-owned subcollections, observer/subscription through `onSnapshot`, optimistic UI updates, query-shaped documents, and async data handling with `async / await`.

What is the difference between Firestore and Redux?

Firestore is the remote database. Redux is local app state. Firestore persists and syncs data. Redux makes the data easy and fast for screens to read while the app is running.